

Contents

1	Mathematical Foundations for Perspective Transformation	2
1.1	Table of Contents	2
1.2	Introduction	3
1.2.1	Purpose	3
1.2.2	Why Perspective Transformation for Autonomous Driving?	3
1.3	Homogeneous Coordinates	3
1.3.1	Motivation: Why Do We Need Them?	3
1.3.2	Definition	3
1.3.3	Conversion Between Representations	4
1.3.4	Special Cases and Properties	4
1.3.5	Translation in Homogeneous Coordinates	5
1.3.6	Summary	5
1.4	Homography Matrix Theory	5
1.4.1	Definition	5
1.4.2	Degrees of Freedom	6
1.4.3	Homography Properties	6
1.4.4	Relationship to Other Transformations	6
1.4.5	Mathematical Formulation	7
1.4.6	Example: Identity Homography	7
1.4.7	Example: Pure Translation	8
1.4.8	Summary	8
1.5	Direct Linear Transformation (DLT) Algorithm	8
1.5.1	Problem Statement	8
1.5.2	Mathematical Derivation	9
1.5.3	Algorithm Summary	10
1.5.4	Minimum Number of Points	11
1.5.5	Degenerate Configurations	11
1.5.6	Numerical Stability Considerations	11
1.5.7	Example: Computing Homography for 4 Points	11
1.5.8	Summary	12
1.6	Inverse Perspective Mapping (IPM) Mathematics	12
1.6.1	The Perspective Projection Problem	12
1.6.2	The Flat Ground Plane Assumption	13
1.6.3	Defining Source and Destination Regions	13
1.6.4	Computing the IPM Homography	14
1.6.5	Forward Transformation: Image $\rightarrow$ BEV	15
1.6.6	Inverse Transformation: BEV $\rightarrow$ Image	15
1.6.7	Metric BEV: Real-World Scale	16
1.6.8	Mathematical Validation	16
1.6.9	Summary	16
1.7	Camera Models	17
1.7.1	Pinhole Camera Model	17
1.7.2	Intrinsic Parameters	17
1.7.3	Extrinsic Parameters	18
1.7.4	Complete Projection: 3D World $\rightarrow$ 2D Image	18
1.7.5	Example: Projection of a 3D Point	19

1.7.6	Backprojection: 2D Image $\rightarrow$ 3D World (with Constraint)	19
1.7.7	Camera Calibration	20
1.7.8	Summary	20
1.8	Coordinate Systems	20
1.8.1	Overview of Coordinate Systems	21
1.8.2	1. World Coordinate System	21
1.8.3	2. Camera Coordinate System	21
1.8.4	3. Image Coordinate System	21
1.8.5	4. BEV Coordinate System	22
1.8.6	Transformation Pipeline Summary	22
1.8.7	Coordinate System Conventions	22
1.8.8	Example: Full Coordinate Transformation	22
1.8.9	Summary	24
1.9	Mathematical Limitations	24
1.9.1	1. Flat Ground Plane Assumption	24
1.9.2	2. Single Homography Limitation	25
1.9.3	3. Numerical Stability Issues	25
1.9.4	4. Distortion and Field of View	25
1.9.5	5. Ambiguity in Depth	26
1.9.6	6. Error Propagation	26
1.9.7	7. Occlusion and Visibility	26
1.9.8	Summary of Limitations	27
1.10	References	27
1.10.1	Essential Papers	27
1.10.2	Books	28
1.10.3	Online Resources	28
1.10.4	Advanced Topics	28

1 Mathematical Foundations for Perspective Transformation

$\Rightarrow$ **Complete mathematical theory for homography-based transformations and Inverse Perspective Mapping (IPM)**

This document provides the mathematical foundations necessary to understand and implement perspective transformations for converting camera images to Bird's Eye View (BEV) representation in autonomous driving applications.

1.1 Table of Contents

1. Introduction
2. Homogeneous Coordinates
3. Homography Matrix Theory
4. Direct Linear Transformation (DLT) Algorithm
5. Inverse Perspective Mapping (IPM) Mathematics
6. Camera Models
7. Coordinate Systems
8. Mathematical Limitations

1.2 Introduction

1.2.1 Purpose

Perspective transformations are fundamental to computer vision and autonomous driving. This document covers the mathematical theory behind:

- **Homography:** Planar projective transformations between image planes
- **Inverse Perspective Mapping (IPM):** Converting front-view camera images to Bird's Eye View
- **Camera Models:** Relating 3D world to 2D image coordinates

1.2.2 Why Perspective Transformation for Autonomous Driving?

In autonomous driving, cameras capture the world in **perspective view**: - Parallel lines (e.g., lane markings) appear to converge toward a vanishing point - Objects farther away appear smaller - Equal distances appear unequal in the image

Bird's Eye View (BEV) provides an **orthographic projection**: - Parallel lines remain parallel - Uniform metric scale (e.g., 1 pixel = 0.1 meters) - Simpler geometric relationships for path planning and lane detection

The Challenge: Transform between these representations using mathematical mappings.

1.3 Homogeneous Coordinates

1.3.1 Motivation: Why Do We Need Them?

In **Cartesian coordinates**, we represent 2D points as (x, y) .

Problem: We cannot represent **translation** as a matrix multiplication in 2D:

Translation: $(x, y) \rightarrow (x + tx, y + ty)$

This is an affine operation, not a linear one. We'd need:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} tx \\ ty \end{bmatrix}$$

The $+ [tx, ty]$ part prevents us from using a single matrix multiplication.

Solution: Homogeneous coordinates add an extra dimension to enable **all** transformations (including translation) as matrix multiplications.

1.3.2 Definition

A 2D Cartesian point (x, y) is represented in **homogeneous coordinates** as a 3D vector:

$$(x, y) \rightarrow (x, y, w) \text{ where } w \neq 0$$

The relationship between Cartesian and homogeneous coordinates is:

$$x = x / w$$

$$y = y / w$$

Key Insight: Multiple homogeneous representations map to the same Cartesian point:

$$(x, y, 1) \sim (2x, 2y, 2) \sim (kx, ky, k) \text{ for any } k \neq 0$$

This is called **projective equivalence**: $(x, y, w) \sim (kx, ky, kw)$.

1.3.3 Conversion Between Representations

1.3.3.1 Cartesian $\rightarrow$ Homogeneous

For a 2D point (x, y) , the standard homogeneous representation uses $w = 1$:

$$(x, y) \rightarrow (x, y, 1)$$

Example:

$$(5, 10) \rightarrow (5, 10, 1)$$

$$(100, 200) \rightarrow (100, 200, 1)$$

1.3.3.2 Homogeneous $\rightarrow$ Cartesian

For a homogeneous point (x, y, w) with $w \neq 0$:

$$(x, y, w) \rightarrow (x/w, y/w)$$

Examples:

$$(10, 20, 2) \rightarrow (10/2, 20/2) = (5, 10)$$

$$(50, 100, 5) \rightarrow (50/5, 100/5) = (10, 20)$$

$$(6, 9, 3) \rightarrow (6/3, 9/3) = (2, 3)$$

1.3.4 Special Cases and Properties

1.3.4.1 Points at Infinity

When $w = 0$, the point represents a **direction** or **point at infinity**:

$$(x, y, 0) \rightarrow (x/0, y/0) = (\infty, \infty) \text{ in direction } (x, y)$$

Geometric Interpretation: - Parallel lines in Cartesian space intersect at infinity in projective space
- All parallel lines with the same direction share the same point at infinity

Example:

$$(1, 0, 0) = \text{point at infinity in the x-direction}$$

$$(0, 1, 0) = \text{point at infinity in the y-direction}$$

1.3.4.2 Invalid Point

The point $(0, 0, 0)$ is **undefined** in homogeneous coordinates (no valid Cartesian equivalent).

1.3.5 Translation in Homogeneous Coordinates

Now we can represent **translation** as a matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

For $w = 1$:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + tx \\ y + ty \\ 1 \end{bmatrix}$$

This unification allows us to represent **ALL** transformations (rotation, scaling, translation, perspective) as **3→3** matrices!

1.3.6 Summary

Property	Cartesian (2D)	Homogeneous (3D)
Representation	(x, y)	(x, y, w) with $w \neq 0$ Equivalence Unique $(x, y, w) \sim (kx, ky, kw)$ Translation Requires addition Matrix multiplication Points at infinity Cannot represent $(x, y, 0)$ 3D (projective plane)
Dimension	2D	3D (projective plane)

Key Takeaway: Homogeneous coordinates enable all geometric transformations as matrix operations, which is essential for perspective transformations and homography.

1.4 Homography Matrix Theory

1.4.1 Definition

A **homography** (also called **projective transformation** or **planar homography**) is a mapping between two **planar** surfaces. It is represented by a **3→3** matrix **H** that transforms homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Where:

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

After applying **H**, we convert back to Cartesian coordinates:

$$x' = (h_{11}x + h_{12}y + h_{13}) / (h_{31}x + h_{32}y + h_{33})$$

$$y' = (h_{21}x + h_{22}y + h_{23}) / (h_{31}x + h_{32}y + h_{33})$$

1.4.2 Degrees of Freedom

The homography matrix has **9 elements**, but only **8 degrees of freedom (DOF)**.

Why 8 DOF? - The matrix is defined up to a scale factor: $H \sim kH$ for any $k \neq 0$ - We can normalize one element (typically $h_{33} = 1$) without loss of generality - This leaves 8 independent parameters

Consequence: We need at least **4 point correspondences** to solve for H : - Each point correspondence gives 2 equations (one for x , one for y) - 4 points $\rightarrow$ 2 equations/point = 8 equations for 8 unknowns

1.4.3 Homography Properties

1.4.3.1 1. Preserves Collinearity

Straight lines remain straight under homography.

If three points are collinear (lie on a line) in the source plane, their transformed points are also collinear in the destination plane.

Mathematical Proof: A line in homogeneous coordinates: $l^T \rightarrow p = 0$ After transformation: $l'^T \rightarrow p' = l'^T \rightarrow H \rightarrow p = 0$ The transformed line $l' = H^{-T} l$ is still a line.

1.4.3.2 2. Preserves Cross-Ratio

The **cross-ratio** of four collinear points is preserved under homography. This is a fundamental projective invariant.

1.4.3.3 3. Does NOT Preserve

- **Distances:** Equal distances in source $\neq$ equal distances in destination
- **Angles:** Right angles may not remain right angles
- **Parallelism:** Parallel lines may not remain parallel (except affine transformations)
- **Areas:** Area ratios change

1.4.3.4 4. Planar Restriction

Critical Assumption: Homography only works for **planar surfaces** or **pure camera rotation**.

If the scene is not planar (e.g., objects at different depths), a single homography cannot correctly map all points.

1.4.4 Relationship to Other Transformations

Homography is the most general 2D transformation. It includes all simpler transformations as special cases:

Transformation	DOF	Matrix Form	Properties Preserved
Euclidean (rotation + translation)	3	$h_{31}=h_{32}=0, h_{33}=1,$ $h_{11}=h_{22}=\cos(\rightarrow),$ $h_{12}=-h_{21}=\sin(\rightarrow)$	Distances, angles, parallelism
Similarity (Euclidean + uniform scale)	4	$h_{31}=h_{32}=0, h_{33}=1,$ isotropic scaling	Angles, parallelism, shape
Affine (similarity + non-uniform scale + shear)	6	$h_{31}=h_{32}=0, h_{33}=1$	Parallelism, area ratios
Projective (Homography)	8	General 3→3 matrix	Collinearity, cross-ratio

Hierarchy:

Euclidean → Similarity → Affine → Projective (Homography)

1.4.5 Mathematical Formulation

Given a set of point correspondences $\{(x_i, y_i) \rightarrow (x'_i, y'_i)\}$, we want to find $\mathbf{H}$ such that:

$$\begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} \sim \mathbf{H} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

The $\sim$ means **projective equivalence** (equal up to scale).

Expanding:

$$x'_i = (h_{11}x_i + h_{12}y_i + h_{13}) / (h_{31}x_i + h_{32}y_i + h_{33})$$

$$y'_i = (h_{21}x_i + h_{22}y_i + h_{23}) / (h_{31}x_i + h_{32}y_i + h_{33})$$

Cross-multiplying to eliminate division:

$$x'_i(h_{31}x_i + h_{32}y_i + h_{33}) = h_{11}x_i + h_{12}y_i + h_{13}$$

$$y'_i(h_{31}x_i + h_{32}y_i + h_{33}) = h_{21}x_i + h_{22}y_i + h_{23}$$

Rearranging:

$$h_{11}x_i + h_{12}y_i + h_{13} - h_{31}x_i x'_i - h_{32}y_i x'_i - h_{33}x'_i = 0$$

$$h_{21}x_i + h_{22}y_i + h_{23} - h_{31}x_i y'_i - h_{32}y_i y'_i - h_{33}y'_i = 0$$

These are **linear equations** in the 9 elements of $\mathbf{H}$ (treated as a 9→1 vector).

1.4.6 Example: Identity Homography

The **identity homography** leaves all points unchanged:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Effect:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Point correspondences:

$(0, 0) \rightarrow (0, 0)$
 $(1, 0) \rightarrow (1, 0)$
 $(0, 1) \rightarrow (0, 1)$
 $(1, 1) \rightarrow (1, 1)$

1.4.7 Example: Pure Translation

Translate by $(tx, ty) = (5, 10)$:

$$H = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 10 \\ 0 & 0 & 1 \end{bmatrix}$$

Effect:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 10 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + 5 \\ y + 10 \\ 1 \end{bmatrix}$$

Point correspondences:

$(0, 0) \rightarrow (5, 10)$
 $(1, 0) \rightarrow (6, 10)$
 $(0, 1) \rightarrow (5, 11)$

1.4.8 Summary

- **Homography** is a $3 \rightarrow 3$ matrix representing planar projective transformation
- **8 degrees of freedom** $\rightarrow$ need **4 point correspondences** minimum
- **Preserves:** straight lines, cross-ratios
- **Does NOT preserve:** distances, angles, parallelism (in general)
- **Only valid for planar scenes** or pure rotation

Next, we'll see how to **compute** the homography matrix from point correspondences using the **Direct Linear Transformation (DLT)** algorithm.

1.5 Direct Linear Transformation (DLT) Algorithm

The **Direct Linear Transformation (DLT)** is the standard algorithm for computing a homography matrix **H** from a set of point correspondences.

1.5.1 Problem Statement

Given: - $N \geq 4$ point correspondences: $\{(x_i, y_i) \rightarrow (x'_i, y'_i)\}$ for $i = 1, \dots, N$ - Source points: (x_i, y_i) in the first image/plane - Destination points: (x'_i, y'_i) in the second image/plane

Find: - The $3 \rightarrow 3$ homography matrix $\mathbf{H}$ such that: $\begin{bmatrix} x'i \\ y'i \\ 1 \end{bmatrix} \sim \mathbf{H} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$

1.5.2 Mathematical Derivation

1.5.2.1 Step 1: Formulate the Constraint Equations

For each point correspondence $(x_i, y_i) \rightarrow (x'i, y'i)$, we have:

$$\begin{bmatrix} x'i \\ y'i \\ w'i \end{bmatrix} \sim \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

Where:

$$\begin{aligned} x'i &= (h_{11}x_i + h_{12}y_i + h_{13}) / (h_{31}x_i + h_{32}y_i + h_{33}) \\ y'i &= (h_{21}x_i + h_{22}y_i + h_{23}) / (h_{31}x_i + h_{32}y_i + h_{33}) \end{aligned}$$

Let $w'i = h_{31}x_i + h_{32}y_i + h_{33}$. Then:

$$\begin{aligned} x'i \rightarrow w'i &= h_{11}x_i + h_{12}y_i + h_{13} \\ y'i \rightarrow w'i &= h_{21}x_i + h_{22}y_i + h_{23} \end{aligned}$$

Substituting $w'i$:

$$\begin{aligned} x'i(h_{31}x_i + h_{32}y_i + h_{33}) &= h_{11}x_i + h_{12}y_i + h_{13} \\ y'i(h_{31}x_i + h_{32}y_i + h_{33}) &= h_{21}x_i + h_{22}y_i + h_{23} \end{aligned}$$

1.5.2.2 Step 2: Rearrange into Linear Equations

Rearranging:

$$\begin{aligned} h_{11}x_i + h_{12}y_i + h_{13}w_i + h_{21}x_i + h_{22}y_i + h_{23}w_i - h_{31}x_i x'i - h_{32}y_i x'i - h_{33}x'i &= 0 \\ h_{11}x_i + h_{12}y_i + h_{13}w_i + h_{21}x_i + h_{22}y_i + h_{23}w_i - h_{31}x_i y'i - h_{32}y_i y'i - h_{33}y'i &= 0 \end{aligned}$$

In matrix form, define the vector of unknown homography elements:

$$\mathbf{h} = [h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33}]^T \quad (9 \times 1 \text{ vector})$$

For each point correspondence, we get **two equations**:

Equation 1 (x-coordinate):

$$[x_i \ y_i \ 1 \ 0 \ 0 \ 0 \ -x_i x'i \ -y_i x'i \ -x'i] \rightarrow \mathbf{h} = 0$$

Equation 2 (y-coordinate):

$$[0 \ 0 \ 0 \ x_i \ y_i \ 1 \ -x_i y'i \ -y_i y'i \ -y'i] \rightarrow \mathbf{h} = 0$$

1.5.2.3 Step 3: Build the System Matrix $\mathbf{A}$

Stack all constraint equations into a matrix $\mathbf{A}$:

For $\mathbf{N}$ point correspondences, we have **2N** equations:

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1 x'_1 & -y_1 x'_1 & -x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1 y'_1 & -y_1 y'_1 & -y'_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2 x'_2 & -y_2 x'_2 & -x'_2 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 0 & 0 & x_2 & y_2 & 1 & -x_2 y_1' & -y_2 y_1' & -y_1'^2 \\ & & & \dots & & & & & \\ x_N & y_N & 1 & 0 & 0 & 0 & -x_N x_1' & -y_N x_1' & -x_1'^2 \\ 0 & 0 & 0 & x_N & y_N & 1 & -x_N y_1' & -y_N y_1' & -y_1'^2 \end{bmatrix}$$

Dimensions: A is $(2N \rightarrow 9)$

The problem becomes: **Find h such that $A \rightarrow h = 0$**

1.5.2.4 Step 4: Solve Using SVD

This is a **homogeneous linear system**. The solution lies in the **null space** of A .

Singular Value Decomposition (SVD):

$$A = U \Sigma V^T$$

Where: - U : $(2N \rightarrow 2N)$ orthogonal matrix - Σ : $(2N \rightarrow 9)$ diagonal matrix of singular values $\sigma_1 \leq \sigma_2 \leq \dots \leq \sigma_9$ - V : $(9 \rightarrow 9)$ orthogonal matrix

Solution: The vector h that minimizes $\|A \rightarrow h\|$ subject to $\|h\| = 1$ is the **last column of V** (corresponding to the smallest singular value σ_9).

Why the last column of V ? - The columns of V are the eigenvectors of $A^T A$ - The last column corresponds to the smallest eigenvalue (smallest singular value squared) - This minimizes $\|A \rightarrow h\|^2 = h^T (A^T A) \rightarrow h$

In NumPy:

```
U, S, Vt = np.linalg.svd(A)
h = Vt[-1, :] # Last row of V^T = last column of V
```

1.5.2.5 Step 5: Reshape to 3x3 Matrix

Reshape the 9x1 vector h into a 3x3 matrix:

```
H = h.reshape(3, 3)
```

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

1.5.2.6 Step 6: Normalization

By convention, we normalize H so that $h_{33} = 1$ (or $\|H\| = 1$).

Common normalization:

```
H = H / H[2, 2] # Make h33 = 1
```

This removes the scale ambiguity.

1.5.3 Algorithm Summary

Input: $N \geq 4$ point correspondences $\{(x_i, y_i) \rightarrow (x'_i, y'_i)\}$

Output: 3→3 homography matrix **H**

Steps: 1. Build the $2N \rightarrow 9$ matrix **A** from point correspondences 2. Compute SVD: $\mathbf{A} = \mathbf{U} \rightarrow \rightarrow \rightarrow \mathbf{V}^T$ 3. Extract solution: $\mathbf{h} = \mathbf{V}[:, -1]$ (last column of **V**) 4. Reshape to 3→3: $\mathbf{H} = \mathbf{h}.\text{reshape}(3, 3)$ 5. Normalize: $\mathbf{H} = \mathbf{H} / \mathbf{H}[2, 2]$

1.5.4 Minimum Number of Points

Exactly 4 points: - 4 points $\rightarrow$ 2 equations = 8 equations for 8 DOF - **Determined system** (assuming points are in general position)

More than 4 points: - $N > 4$ points $\rightarrow 2N > 8$ equations - **Overdetermined system** - SVD gives the **least-squares solution** that minimizes reprojection error

Fewer than 4 points: - $N < 4 \rightarrow 2N < 8$ equations - **Underdetermined system** - Infinite solutions (cannot uniquely determine **H**)

1.5.5 Degenerate Configurations

Collinear Points: If all points lie on a line, the system is degenerate and has no unique solution.

Three Points: Three non-collinear points define an **affine transformation** (6 DOF), not a full homography (8 DOF).

Good Practice: - Use **N e 4** non-collinear points - Prefer **N > 4** for robustness (least-squares fitting) - Ensure points are **well-distributed** (not clustered)

1.5.6 Numerical Stability Considerations

Data Normalization: For numerical stability, normalize point coordinates before computing homography:

1. **Translate** points so centroid is at origin
2. **Scale** so average distance from origin is 2
3. Compute **H'** from normalized points
4. **Denormalize** to get final **H**

Why? - Large coordinate values can cause numerical instability in SVD - Normalization improves conditioning of matrix **A**

Formula:

$$\mathbf{H} = \mathbf{T}_{\text{dest}}^{-1} \rightarrow \mathbf{H}' \rightarrow \mathbf{T}_{\text{src}}$$

Where $\mathbf{T}_{\text{src}}$ and $\mathbf{T}_{\text{dest}}$ are the normalization transforms for source and destination points.

1.5.7 Example: Computing Homography for 4 Points

Source points:

- $(x_1, y_1) = (0, 0)$
- $(x_2, y_2) = (1, 0)$
- $(x_3, y_3) = (1, 1)$
- $(x_4, y_4) = (0, 1)$

Destination points (2→ scaling):

$$(x'1, y'1) = (0, 0)$$

$$(x'2, y'2) = (2, 0)$$

$$(x'3, y'3) = (2, 2)$$

$$(x'4, y'4) = (0, 2)$$

Matrix A:

[0	0	1	0	0	0	0	0	0]	(x-equation for point 1)
[0	0	0	0	0	1	0	0	0]	(y-equation for point 1)
[1	0	1	0	0	0	-2	-0	-2]	(x-equation for point 2)
[0	0	0	1	0	1	-2	-0	-2]	(y-equation for point 2)
[1	1	1	0	0	0	-2	-2	-2]	(x-equation for point 3)
[0	0	0	1	1	1	-2	-2	-2]	(y-equation for point 3)
[0	1	1	0	0	0	0	-2	-2]	(x-equation for point 4)
[0	0	0	0	1	1	0	-2	-2]	(y-equation for point 4)

SVD solution → h:

$$h \propto [2, 0, 0, 0, 2, 0, 0, 0, 1]^T$$

Homography H:

$$H = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This is a **2→ uniform scaling** transformation, as expected.

1.5.8 Summary

- **DLT** solves for homography using SVD of a linear system
- **Build matrix A** from point correspondences (2 rows per point)
- **Solution** is the last column of V in SVD: $A = U \Sigma V^T$
- **Minimum 4 points** required; more points give least-squares solution
- **Normalization** improves numerical stability

1.6 Inverse Perspective Mapping (IPM) Mathematics

Inverse Perspective Mapping (IPM) transforms a front-view camera image into a **Bird's Eye View (BEV)** representation. This is a specific application of homography with important geometric assumptions.

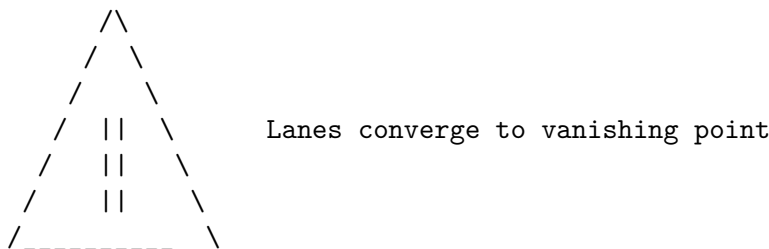
1.6.1 The Perspective Projection Problem

1.6.1.1 Front View (Perspective)

A camera captures a 3D scene as a 2D image through **perspective projection**: - Objects farther away appear smaller - Parallel lines (e.g., road lanes) converge toward a vanishing point - Equal distances on the ground appear unequal in the image

Example: Lane markings on a straight road

Image plane (front view):

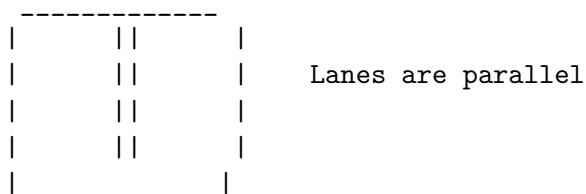


1.6.1.2 Bird's Eye View (Orthographic)

BEV provides an **orthographic projection** from directly above: - Parallel lines remain parallel - Equal distances on the ground map to equal distances in BEV - Uniform metric scale (e.g., 1 pixel = 0.1 meters)

Same lane markings in BEV:

BEV (top-down view):



1.6.2 The Flat Ground Plane Assumption

Critical Assumption: IPM assumes all points lie on a **flat ground plane** at $Z = 0$ in world coordinates.

Mathematical Implication: - 3D world point: $(X, Y, Z = 0)$ - Under this constraint, the mapping from image to ground plane is a **homography**

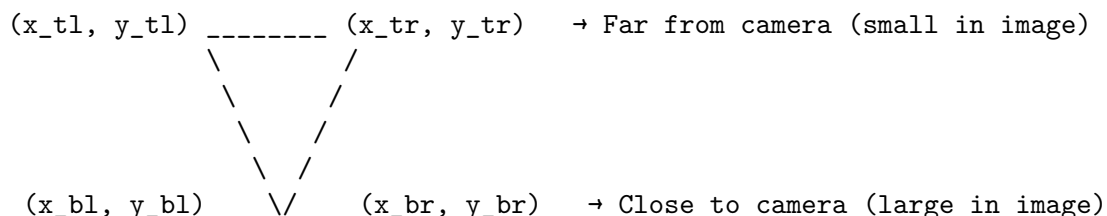
Why? - Both the image plane and ground plane are planar surfaces - Homography maps between two planes - The transformation is uniquely defined by point correspondences

Consequence: Objects **not** on the ground plane (vehicles, pedestrians, overhead signs) will be **incorrectly transformed** and appear distorted in BEV.

1.6.3 Defining Source and Destination Regions

1.6.3.1 Source Region: Trapezoid in Front View

In the front-view image, we define a **region of interest (ROI)** as a trapezoid:



Typical configuration:

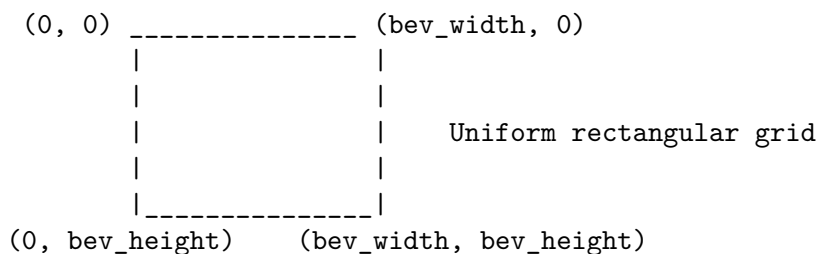
```
# Image dimensions: width + height
# ROI as ratios of image size

top_left      = (width * 0.4, height * 0.6)  # Top-left of trapezoid
top_right     = (width * 0.6, height * 0.6)  # Top-right of trapezoid
bottom_left   = (width * 0.1, height * 0.95) # Bottom-left of trapezoid
bottom_right  = (width * 0.9, height * 0.95) # Bottom-right of trapezoid
```

Rationale: - Top edge (far from camera): narrower width due to perspective - Bottom edge (close to camera): wider width - Covers the road region where lanes are visible

1.6.3.2 Destination Region: Rectangle in BEV

In BEV, we define a **rectangular region**:



Typical configuration:

```
bev_width = 640  # BEV image width in pixels
bev_height = 480 # BEV image height in pixels

dst_points = [
    [0, 0],          # Top-left
    [bev_width, 0],   # Top-right
    [bev_width, bev_height], # Bottom-right
    [0, bev_height]   # Bottom-left
]
```

Correspondence:

Source (trapezoid)	→	Destination (rectangle)
(x_tl, y_tl)	→	(0, 0)
(x_tr, y_tr)	→	(bev_width, 0)
(x_br, y_br)	→	(bev_width, bev_height)
(x_bl, y_bl)	→	(0, bev_height)

1.6.4 Computing the IPM Homography

Using the 4 **point correspondences** between source trapezoid and destination rectangle, we compute the homography **H** using the **DLT algorithm**:

```
H = compute_homography(src_points, dst_points)
```

Where:

```

src_points = np.array([
    [x_tl, y_tl],
    [x_tr, y_tr],
    [x_br, y_br],
    [x_bl, y_bl]
])

dst_points = np.array([
    [0, 0],
    [bev_width, 0],
    [bev_width, bev_height],
    [0, bev_height]
])

```

Homography H:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

1.6.5 Forward Transformation: Image $\rightarrow$ BEV

To transform the entire image:

```
BEV_image = warpPerspective(front_view_image, H, (bev_width, bev_height))
```

For individual points:

$$\begin{bmatrix} x_{bev} \\ y_{bev} \\ w \end{bmatrix} = H \begin{bmatrix} x_{img} \\ y_{img} \\ 1 \end{bmatrix}$$

Then convert from homogeneous:

```

x_bev_cartesian = x_bev / w
y_bev_cartesian = y_bev / w

```

1.6.6 Inverse Transformation: BEV $\rightarrow$ Image

To map points from BEV back to the original image:

```
H_inv = inverse(H)
```

For points:

$$\begin{bmatrix} x_{img} \\ y_{img} \\ w \end{bmatrix} = H_{inv} \rightarrow \begin{bmatrix} x_{bev} \\ y_{bev} \\ 1 \end{bmatrix}$$

For images:

```
front_view_image = warpPerspective(BEV_image, H_inv, (img_width, img_height))
```

1.6.7 Metric BEV: Real-World Scale

To create a **metrically accurate BEV** (e.g., 1 pixel = 0.1 meters), we need camera calibration parameters and ground plane geometry.

Approach: 1. Define real-world ground plane coordinates in meters 2. Project world coordinates to image coordinates using camera model 3. Use these correspondences to compute homography

Example:

World coordinates (meters):		Image coordinates (pixels):
X = -5m, Y = 10m, Z = 0	→ project	(u1, v1)
X = 5m, Y = 10m, Z = 0	→ project	(u2, v2)
X = 5m, Y = 30m, Z = 0	→ project	(u3, v3)
X = -5m, Y = 30m, Z = 0	→ project	(u4, v4)

Then:

BEV coordinates (pixels):
(0, 0), (100, 0), (100, 200), (0, 200)

Homography: `src_points (image) → dst_points (BEV pixels)`

Benefit: - 1 BEV pixel = 0.1 meters in real world - Enables accurate distance measurements - Consistent scale across different camera setups

1.6.8 Mathematical Validation

Round-Trip Test: A point should return to its original location after forward and inverse transformation:

`p_img → (apply H) → p_bev → (apply H_inv) → p_img'`

Ideally: `p_img' ≈ p_img` (within numerical precision)

Parallel Line Test: Parallel lines on the ground (e.g., lane markings) should remain parallel in BEV.

Distance Preservation: Equal distances on the ground plane should map to approximately equal distances in BEV (depends on calibration quality).

1.6.9 Summary

- **IPM** uses homography to transform front-view to BEV
- **Assumption:** Flat ground plane ($Z = 0$)
- **Source:** Trapezoid in front-view image (perspective effect)
- **Destination:** Rectangle in BEV (orthographic view)
- **Homography H** computed from 4 point correspondences
- **Forward:** `BEV = warpPerspective(image, H, size)`
- **Inverse:** `Image = warpPerspective(BEV, H_inv, size)`
- **Metric BEV** requires camera calibration for real-world scale

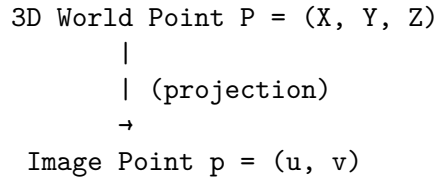
1.7 Camera Models

To fully understand the relationship between 3D world coordinates and 2D image coordinates, we need to model the camera's projection process.

1.7.1 Pinhole Camera Model

The **pinhole camera** is the fundamental model in computer vision. It describes how a 3D point in the world is projected onto a 2D image plane.

Geometry:



Coordinate Systems: 1. **World coordinates:** (X, Y, Z) in meters 2. **Camera coordinates:** (X_c, Y_c, Z_c) relative to camera center 3. **Image coordinates:** (u, v) in pixels

1.7.2 Intrinsic Parameters

Intrinsic parameters describe the camera's **internal geometry**:

1. **Focal length:** f_x, f_y (in pixels)
 - Relationship between 3D distance and 2D projection
 - May differ slightly in x and y due to non-square pixels
2. **Principal point:** (c_x, c_y) (in pixels)
 - Center of the image sensor
 - Typically near the image center: $(\text{width}/2, \text{height}/2)$
3. **Skew coefficient:** s
 - Angle between pixel axes (typically 0 for modern cameras)

Intrinsic Matrix K :

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

For most cameras, skew $s = 0$:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Typical values:

$f_x = f_y$ H 500-1000 pixels (for typical cameras)
 c_x H image_width / 2
 c_y H image_height / 2

1.7.3 Extrinsic Parameters

Extrinsic parameters describe the camera's **position and orientation** in the world:

1. **Rotation:** 3→3 rotation matrix R
 - Orientation of camera frame relative to world frame
 - Orthogonal matrix: $R^T \rightarrow R = I, \det(R) = 1$
2. **Translation:** 3→1 translation vector t
 - Position of camera center in world coordinates
 - Units: meters

Transformation from World to Camera Coordinates:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + t$$

Or in homogeneous coordinates:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & | & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Where $\begin{bmatrix} R & | & t \end{bmatrix}$ is a 3→4 matrix.

1.7.4 Complete Projection: 3D World → 2D Image

Step 1: World to Camera Coordinates

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & | & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Step 2: Perspective Projection

$$\begin{aligned} x &= X_c / Z_c \\ y &= Y_c / Z_c \end{aligned}$$

Step 3: Apply Intrinsic Parameters

$$\begin{aligned} u &= f_x \rightarrow x + c_x = f_x \rightarrow (X_c / Z_c) + c_x \\ v &= f_y \rightarrow y + c_y = f_y \rightarrow (Y_c / Z_c) + c_y \end{aligned}$$

Combined Projection Matrix:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \rightarrow \begin{bmatrix} R & | & t \end{bmatrix} \rightarrow \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Where $\rightarrow = Z_c$ is the depth in camera coordinates.

Projection matrix P :

$$P = K \rightarrow [R \mid t] \quad (3 \times 4 \text{ matrix})$$

Full equation:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = P \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

After projection, divide by $\rightarrow$ to get pixel coordinates (u, v) .

1.7.5 Example: Projection of a 3D Point

Camera parameters:

$$K = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \quad (f_x=800, f_y=800, c_x=320, c_y=240)$$

$$R = I \text{ (identity - no rotation)}$$

$$t = [0, 0, 0]^T \text{ (camera at world origin)}$$

3D point:

$$P = (X, Y, Z) = (1, 2, 5) \text{ meters}$$

Projection:

$$\text{Camera coordinates: } (X_c, Y_c, Z_c) = (1, 2, 5) \quad (\text{same as world, since } R=I, t=0)$$

$$\text{Normalized coordinates: } x = 1/5 = 0.2, \quad y = 2/5 = 0.4$$

Image coordinates:

$$u = 800 \rightarrow 0.2 + 320 = 160 + 320 = 480 \text{ pixels}$$

$$v = 800 \rightarrow 0.4 + 240 = 320 + 240 = 560 \text{ pixels}$$

Result: Point $(1, 2, 5)$ projects to pixel $(480, 560)$.

1.7.6 Backprojection: 2D Image $\rightarrow$ 3D World (with Constraint)

Recovering 3D from 2D is **ambiguous** without additional information: - A single image pixel corresponds to a **ray in 3D space** - Infinite 3D points project to the same 2D pixel (different depths)

Constraint Needed: Assume a known depth or known ground plane.

1.7.6.1 Backprojection to Ground Plane ($Z = 0$)

For IPM, we assume the ground plane at $Z = 0$.

Given: - Image point: (u, v) - Ground plane: $Z = 0$

Unknown: - World coordinates: $(X, Y, 0)$

Solve:

From the projection equation:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = P \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix}$$

This gives 3 equations for 2 unknowns (X, Y) (since Z = 0 is fixed):

$$\begin{aligned} u &= p_{11}X + p_{12}Y + p_{14} \\ v &= p_{21}X + p_{22}Y + p_{24} \\ 1 &= p_{31}X + p_{32}Y + p_{34} \end{aligned}$$

Substitute the third equation into the first two:

$$\begin{aligned} u &- (p_{31}X + p_{32}Y + p_{34}) = p_{11}X + p_{12}Y + p_{14} \\ v &- (p_{31}X + p_{32}Y + p_{34}) = p_{21}X + p_{22}Y + p_{24} \end{aligned}$$

This is a **linear system in (X, Y)**:

$$\begin{bmatrix} p_{11} - u p_{31} & p_{12} - u p_{32} \\ p_{21} - v p_{31} & p_{22} - v p_{32} \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} u p_{34} - p_{14} \\ v p_{34} - p_{24} \end{bmatrix}$$

Solve for (X, Y) using matrix inversion or pseudo-inverse.

Result: The 3D ground plane coordinates (X, Y, 0) corresponding to image pixel (u, v).

1.7.7 Camera Calibration

Calibration determines the intrinsic K and extrinsic [R | t] parameters.

Method: Use a known calibration pattern (e.g., checkerboard) 1. Capture images of the checkerboard from multiple angles 2. Detect corner points in the images 3. Solve for camera parameters using correspondences 4. Tools: OpenCV `calibrateCamera()` function

Output: - Intrinsic matrix K - Distortion coefficients (lens distortion) - Extrinsic parameters [R | t] for each image

1.7.8 Summary

- **Pinhole camera model** describes 3D $\rightarrow$ 2D projection
- **Intrinsic parameters (K):** focal length (fx, fy), principal point (cx, cy)
- **Extrinsic parameters:** rotation R, translation t
- **Projection matrix:** $P = K \rightarrow [R | t]$
- **Projection:** (X, Y, Z) $\rightarrow$ (u, v) using P
- **Backprojection:** (u, v) $\rightarrow$ (X, Y, Z) requires additional constraint (e.g., Z = 0)
- **Calibration** determines camera parameters from known patterns

1.8 Coordinate Systems

Understanding different coordinate systems is crucial for IPM and autonomous driving applications.

1.8.1 Overview of Coordinate Systems

Four main coordinate systems are involved:

1. **World Coordinates (3D):** (X, Y, Z) in meters
2. **Camera Coordinates (3D):** (X_c, Y_c, Z_c) relative to camera
3. **Image Coordinates (2D):** (u, v) in pixels
4. **BEV Coordinates (2D):** (x_{bev}, y_{bev}) in pixels (top-down view)

1.8.2 1. World Coordinate System

Definition: - **Origin:** User-defined reference point (e.g., vehicle center, map origin) - **Axes:** - **X:** Forward direction (typically front of vehicle) - **Y:** Left direction (perpendicular to X) - **Z:** Upward direction (perpendicular to ground) - **Units:** Meters

Usage: Represents positions in the real world.

Example:

Vehicle coordinate frame:

```
      Z (up)
      |
      |
      |----- X (forward)
     /
    /
   Y (left)
```

1.8.3 2. Camera Coordinate System

Definition: - **Origin:** Camera optical center - **Axes:** - **X_c:** Right (perpendicular to optical axis) - **Y_c:** Down (perpendicular to optical axis) - **Z_c:** Forward (optical axis, into the scene) - **Units:** Meters

Transformation from World:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \mathbf{t}$$

Where R is rotation matrix, $\mathbf{t}$ is translation vector.

Usage: Intermediate step for camera projection.

1.8.4 3. Image Coordinate System

Definition: - **Origin:** Top-left corner of the image (typically) - **Axes:** - **u:** Horizontal (left to right), x-axis - **v:** Vertical (top to bottom), y-axis - **Units:** Pixels

Transformation from Camera:

$$\begin{aligned} u &= f_x \rightarrow (X_c / Z_c) + c_x \\ v &= f_y \rightarrow (Y_c / Z_c) + c_y \end{aligned}$$

Where (f_x, f_y) are focal lengths, (c_x, c_y) is principal point.

Usage: Pixel coordinates in the captured image.

Note: Some conventions use origin at center or bottom-left. OpenCV uses top-left.

1.8.5 4. BEV Coordinate System

Definition: - **Origin:** User-defined (e.g., vehicle rear axle projected onto ground) - **Axes:** - x_{bev} : Horizontal in BEV (typically aligned with vehicle lateral direction) - y_{bev} : Vertical in BEV (typically aligned with vehicle forward direction) - **Units:** Pixels (or meters for metric BEV)

Transformation from Image:

$$\begin{bmatrix} x_{bev} \\ y_{bev} \\ w \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Where H is the IPM homography matrix.

Usage: Top-down view for planning and perception.

1.8.6 Transformation Pipeline Summary

3D World $\rightarrow$ Image:

$$(X, Y, Z) \rightarrow [\text{extrinsics } R, t] \rightarrow (X_c, Y_c, Z_c) \rightarrow [\text{intrinsics } K] \rightarrow (u, v)$$

Image $\rightarrow$ BEV:

$$(u, v) \rightarrow [\text{homography } H] \rightarrow (x_{bev}, y_{bev})$$

Complete Pipeline (World $\rightarrow$ BEV):

$$(X, Y, Z=0) \rightarrow [R, t, K] \rightarrow (u, v) \rightarrow [H] \rightarrow (x_{bev}, y_{bev})$$

Reverse (BEV $\rightarrow$ World):

$$(x_{bev}, y_{bev}) \rightarrow [H_{inv}] \rightarrow (u, v) \rightarrow [\text{backproject with } Z=0] \rightarrow (X, Y, 0)$$

1.8.7 Coordinate System Conventions

Right-Handed vs. Left-Handed: Most systems use **right-handed** coordinate systems: - $X \rightarrow Y = Z$ (cross product)

Direction Conventions: - **World (vehicle-centric):** X =forward, Y =left, Z =up - **Camera (computer vision):** Z_c =forward (optical axis), X_c =right, Y_c =down - **Image (OpenCV):** u =right, v =down, origin at top-left

Always verify the coordinate system convention for your specific application!

1.8.8 Example: Full Coordinate Transformation

Setup:

World point: $P = (5, 2, 0)$ meters (5m forward, 2m left, on ground)

Camera extrinsics:

$R = I$ (no rotation)

$t = (0, 0, 1.5)$ (camera 1.5m above ground at vehicle origin)

Camera intrinsics:

$f_x = f_y = 800$

$c_x = 320, c_y = 240$

Step 1: World $\rightarrow$ Camera

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = I \begin{bmatrix} 5 \\ 2 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ 1.5 \end{bmatrix}$$

Wait, this is incorrect. The translation t represents the camera position in the world. The transformation should be:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R^T \rightarrow ([Y] - [t_y]) = I \rightarrow \begin{bmatrix} 5 - 0 \\ 2 - 0 \\ 0 - 1.5 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ -1.5 \end{bmatrix}$$

Actually, for camera coordinates, we need:

$$P_c = R \rightarrow (P_w - t)$$

But since $R = I$:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} 5 - 0 \\ 2 - 0 \\ 0 - 1.5 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ -1.5 \end{bmatrix}$$

Hmm, negative Z_c means the point is behind the camera (not visible). Let me reconsider.

Correction: If the camera is at $(0, 0, 1.5)$ meters (above ground), and we want to project a ground point $(5, 2, 0)$:

The camera looks in the $+X$ direction (forward). The camera frame: - Origin at $(0, 0, 1.5)$ - Z_c axis pointing forward ($+X$ in world) - Y_c axis pointing down ($-Z$ in world)

For simplicity, assume:

R rotates from world to camera:

- World $+X \rightarrow$ Camera $+Z_c$
- World $+Y \rightarrow$ Camera $-X_c$
- World $+Z \rightarrow$ Camera $-Y_c$

Approximation:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} -2 \\ 1.5 \\ 5 \end{bmatrix} \quad (\text{point is 5m forward, 2m left} \rightarrow \text{in camera frame})$$

Step 2: Camera $\rightarrow$ Image

$$u = fx \rightarrow (X_c / Z_c) + c_x = 800 \rightarrow (-2 / 5) + 320 = 800 \rightarrow (-0.4) + 320 = -320 + 320 = 0$$

$$v = fy \rightarrow (Y_c / Z_c) + c_y = 800 \rightarrow (1.5 / 5) + 240 = 800 \rightarrow 0.3 + 240 = 240 + 240 = 480$$

Image coordinates: (u, v) H (0, 480) pixels

Step 3: Image $\rightarrow$ BEV

Assume we have precomputed homography **H** from image to BEV:

$$\begin{bmatrix} x_{bev} \\ y_{bev} \\ w \end{bmatrix} = H \begin{bmatrix} 0 \\ 480 \\ 1 \end{bmatrix}$$

After division by **w**, we get BEV coordinates.

This example illustrates the complexity of coordinate transformations - careful book-keeping is essential!

1.8.9 Summary

- **Four coordinate systems:** World (3D), Camera (3D), Image (2D), BEV (2D)
- **World $\rightarrow$ Camera:** Extrinsic parameters $[R \mid t]$
- **Camera $\rightarrow$ Image:** Intrinsic parameters **K**
- **Image $\rightarrow$ BEV:** Homography **H**
- **Careful attention** to coordinate frame conventions is critical

1.9 Mathematical Limitations

Understanding the limitations of homography and IPM is crucial for robust system design.

1.9.1 1. Flat Ground Plane Assumption

Assumption: All points lie on a plane ($Z = 0$ in world coordinates).

Violation: Non-planar surfaces (hills, slopes, speed bumps).

Effect: - Homography is **exact only for planar scenes** - Points **not** on the plane will be **incorrectly projected** - Error increases with distance from the plane

Mathematical Reason: Homography relates two **planes**, not arbitrary 3D surfaces. For a point at height **h** above ground:

True world: (X, Y, Z = h)

IPM assumes: (X, Y, Z = 0)

The projection will be incorrect by an amount proportional to **h**.

Example: - **Pedestrian (height ~1.8m):** Severely distorted in BEV - **Vehicle (height ~1.5m):** Appears stretched and displaced - **Overhead sign:** May appear on the ground

Mitigation: - Restrict IPM to known flat regions (road surface) - Use semantic segmentation to mask non-ground objects - Consider multi-plane or 3D reconstruction for non-flat scenes

1.9.2 2. Single Homography Limitation

Assumption: A single homography H applies to the entire ROI.

Violation: Curved roads, varying ground plane orientation.

Effect: - On **curved roads**, a single trapezoid cannot accurately capture the road geometry - Different parts of the scene may require different homographies

Mathematical Reason: A homography is a **global transformation**. For curved surfaces or non-uniform planes, a single homography cannot adapt locally.

Example: - **Curved highway exit ramp:** Lanes appear distorted at the curve - **Varying pitch (uphill/downhill):** Ground plane changes orientation

Mitigation: - **Piecewise IPM:** Divide image into multiple regions, each with its own homography - **Polynomial or spline fitting:** Use higher-order transformations - **Adaptive homography:** Recompute H based on detected road curvature

1.9.3 3. Numerical Stability Issues

1.9.3.1 Condition Number

The **condition number** of matrix A (in DLT) measures how sensitive the solution is to perturbations:

$$\kappa(A) = \sigma_{\max} / \sigma_{\min}$$

Where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values.

High condition number ($\kappa \gg 1$): - **Ill-conditioned system** - Small errors in point correspondences $\rightarrow$ large errors in H - Unstable solution

Causes: - **Collinear or nearly collinear points** - **Clustered points** (poor spatial distribution) - **Large magnitude differences** in coordinates

Mitigation: - **Normalize coordinates** before DLT (shift + scale) - **Well-distributed points** spanning the entire ROI - **More than 4 points** for overdetermined least-squares fitting

1.9.3.2 Degenerate Point Configurations

Collinear Points: All points on a single line $\rightarrow$ Infinite solutions for H .

3 Points: Defines an affine transformation (6 DOF), not a full homography (8 DOF).

Coincident Points: Duplicate points $\rightarrow$ Rank-deficient matrix A .

Good Practice: - Use **4+ non-collinear points** - Ensure points span **2D space** (not confined to a line) - Check matrix rank before solving

1.9.4 4. Distortion and Field of View

Lens Distortion: Real cameras have **radial and tangential distortion:** - Barrel distortion (straight lines appear curved outward) - Pincushion distortion (straight lines appear curved inward)

Effect on IPM: - Homography assumes a **perfect pinhole camera** (no distortion) - Distortion causes errors in point correspondences - BEV will have geometric errors

Mitigation: - **Undistort image** before applying IPM using calibration parameters - Use OpenCV `undistort()` with camera matrix and distortion coefficients

Wide Field of View: - Extreme wide-angle lenses (fisheye) have severe distortion - Standard pinhole + distortion model may not suffice - Consider specialized fisheye models

1.9.5 5. Ambiguity in Depth

Problem: 2D image $\rightarrow$ 3D world is **inherently ambiguous**.

Mathematical:

Image point $(u, v) \rightarrow$ Ray in 3D space

All points along the ray project to the same pixel

IPM Solution: Assume $Z = 0$ (ground plane).

Limitation: Cannot distinguish objects at different heights.

Effect: - All objects mapped to ground plane in BEV - Occlusions and layering information lost

Mitigation: - Use **stereo cameras** or **LiDAR** for depth information - Semantic segmentation to identify elevated objects - Multi-view geometry for 3D reconstruction

1.9.6 6. Error Propagation

Sources of Error: 1. **Point correspondence errors:** Manual selection, feature detection inaccuracy 2. **Camera calibration errors:** Inaccurate intrinsic/extrinsic parameters 3. **Numerical errors:** Floating-point precision, ill-conditioned matrices 4. **Model errors:** Distortion, ground plane assumption violations

Propagation: Errors compound through the transformation pipeline:

World $\rightarrow$ Camera $\rightarrow$ Image $\rightarrow$ BEV

Each stage accumulates error, which is amplified by subsequent transformations.

Quantification: - **Reprojection error:** Distance between observed and predicted image points -

Homography error: $|| H \rightarrow \text{src} - \text{dst} ||$ for correspondences - **BEV error:** Deviation from ground truth measurements

Mitigation: - **High-quality calibration** - **Robust estimation** (RANSAC for outlier rejection) - **Validation** with known ground truth - **Redundancy:** More correspondences than minimum required

1.9.7 7. Occlusion and Visibility

Problem: Not all ground plane points are visible in the image.

Causes: - **Occlusion** by vehicles, pedestrians, barriers - **Field of view limits** - **Image boundaries**

Effect on IPM: - Some BEV pixels have no corresponding image data - Appears as **black or undefined regions** in BEV

Mitigation: - Define **ROI** to match visible ground plane region - **Multi-camera fusion** for 360° coverage - **Interpolation or inpainting** for missing regions (use with caution)

1.9.8 Summary of Limitations

Limitation	Cause	Effect	Mitigation
Non-planar scenes	Z ≠ 0 assumption	Distortion of elevated objects	Semantic masking, 3D methods
Curved roads	Single homography	Geometric distortion	Piecewise IPM, adaptive H
Numerical instability	Ill-conditioned A	Inaccurate H	Normalization, distributed points
Lens distortion	Real camera optics	BEV geometric errors	Undistort image first
Depth ambiguity	2D → 3D mapping	Cannot resolve height	Stereo, LiDAR, semantic info
Error propagation	Cascaded transformations	Accumulated inaccuracy	Quality calibration, validation
Occlusion	Limited FOV, obstacles	Missing BEV regions	Multi-camera, careful ROI

Key Insight: IPM is **powerful but constrained**. Understanding these limitations is essential for:
- Knowing when IPM is appropriate - Designing robust systems - Validating results - Choosing alternative methods when needed

1.10 References

1.10.1 Essential Papers

1. Mallot, H. A., et al. (1991). "Inverse Perspective Mapping Simplifies Optical Flow Computation and Obstacle Detection." *Biological Cybernetics*, 64(3), 177-185.
 - Original IPM formulation
2. Bertozzi, M., & Broggi, A. (1998). "GOLD: A Parallel Real-Time Stereo Vision System for Generic Obstacle and Lane Detection." *IEEE Transactions on Image Processing*, 7(1), 62-81.
 - IPM for lane detection
3. Hartley, R., & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
 - **Chapter 4:** Homography estimation and DLT algorithm
 - **Chapter 2:** Projective geometry and homogeneous coordinates
4. Zhang, Z. (2000). "A Flexible New Technique for Camera Calibration." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), 1330-1334.
 - Camera calibration method (widely used)

1.10.2 Books

5. **Szeliski, R. (2022).** *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
 - **Chapter 2:** Image formation and camera models
 - **Chapter 6:** Feature-based alignment and homographies
6. **Forsyth, D. A., & Ponce, J. (2011).** *Computer Vision: A Modern Approach* (2nd ed.). Pearson.
 - **Chapter 3:** Geometric camera models

1.10.3 Online Resources

7. **OpenCV Documentation**
 - `cv2.findHomography()`: https://docs.opencv.org/master/d9/d0c/group___calib3d.html
 - `cv2.warpPerspective()`: https://docs.opencv.org/master/da/d54/group___imgproc___transform.html
8. **First Principles of Computer Vision (YouTube)**
 - Shree Nayar’s lecture series on homography and projective geometry
9. **Cyrill Stachniss - Photogrammetry Lectures**
 - YouTube series covering DLT, camera models, and transformations

1.10.4 Advanced Topics

10. **Phillon, J., & Fidler, S. (2020).** “Lift, Splat, Shoot: Encoding Images from Arbitrary Camera Rigs by Implicitly Unprojecting to 3D.” *ECCV 2020*.
 - Modern learned approach to BEV transformation
11. **Li, Z., et al. (2022).** “BEVFormer: Learning Bird’s-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers.” *ECCV 2022*.
 - Transformer-based BEV for autonomous driving

End of Mathematical Foundations Document

This document provides the mathematical theory needed to understand and implement perspective transformations and Inverse Perspective Mapping. For practical implementation details, see `implementation_notes.md`.